TRAMA · MANI — Infraestructura

Proyecto: MANI — Plataforma Multi-Tenant de Formalización de Operaciones de Servicio
Documento: Infraestructura y Topología de Ambientes
Responsable principal: DevOps
Documento vivo: sin número de versión; la vigente es la de `main`. Las decisiones abiertas están explícitas en §25.

---

1. Propósito

Este documento describe **dónde y cómo se ejecutan los componentes de MANI**, qué se comparte, qué se aísla entre ambientes y qué decisiones de infraestructura permanecen abiertas.

No redefine:

- arquitectura funcional → SAD;
- diseño interno → SDD;
- políticas CI/CD → Políticas DevOps;
- DDL → Modelo de Datos;
- decisiones históricas → ADR.

---

2. Principios de infraestructura

1. Tres ambientes oficiales, y son exactamente tres: **DEV → QA → PROD**.
2. **DEV corre en las máquinas personales** de cada desarrollador, con Docker local.
3. **QA y PROD corren en una máquina virtual por ambiente, con Docker.**
4. Datos y secretos aislados por ambiente.
5. Artefactos OCI versionados e inmutables, fijados por tag y digest.
6. Registro centralizado en GHCR.
7. Configuración externa a las imágenes.
8. Misma definición de esquema y RLS entre ambientes.
9. Supabase como plataforma administrada de datos.
10. PostgreSQL como motor de base de datos.
11. **La plataforma de orquestación es una decisión abierta.** Kubernetes es el objetivo exigido por PROY-08, pero no es el estado actual (§10).
12. Ningún dato real de PROD se copia a DEV o QA.

---

3. Vista general



```
   ├── Dispatch Service ..... .NET     · MANI-Dispatch-Service
   └── Core Service ......... Node.js  · MANI-Core-Service
            │
            ▼
        Supabase
        ├── PostgreSQL
        ├── Auth
        ├── Storage
        └── Realtime
```

Tres servicios de negocio. La cobertura y la disponibilidad son un dominio del Core Service, con sus tablas en el esquema `core`. Los dos únicos caminos del cliente a Supabase son Auth y Realtime (SAD §8.3).

Servicios externos:

- FCM / APNs;
- operador de pagos en segundo incremento;
- Prometheus;
- Grafana;
- Datadog;
- Data Warehouse.

---

4. Ambientes oficiales

```
DEV → QA → PROD
```

Ambiente	Dónde corre	Plataforma	Datos
DEV	máquina personal de cada desarrollador	Docker local + Supabase de DEV	sintéticos
QA	VM de QA	Docker + Supabase de QA	dataset controlado y anonimizado
PROD	VM productiva	Docker + Supabase productivo	reales

`QA` es un único ambiente y se llama **QA** en todo documento, pipeline y tag. No se usan «TEST», «QA» ni «Staging»: `Staging` no constituye un cuarto ambiente oficial y no existe.

---

5. Ambiente DEV

5.1 Objetivo

DEV se utiliza para:

- desarrollo;
- integración temprana;
- pruebas locales;
- validación de esquema;
- pruebas funcionales antes de QA.

5.2 Supabase DEV compartido

Existe un ambiente de **Supabase para DEV**, administrado por el equipo, equivalente conceptualmente a los proyectos utilizados para QA y PROD.

Debe conservar:

- PostgreSQL;
- esquema versionado;
- Auth;
- Storage;
- Realtime;
- RLS;
- datos exclusivamente de desarrollo.

La administración actual del ambiente DEV corresponde a Nicolás León.

5.3 Desarrollo en máquinas personales

DEV no tiene servidor propio: corre en la máquina personal de cada desarrollador, con Docker local y el mismo esquema versionado. El Supabase de DEV compartido (§5.2) es el que aporta Auth, Storage, Realtime y RLS a esas máquinas.

Que DEV sea local tiene dos consecuencias que no son defectos, sino el modelo elegido:

- no existe un «ambiente DEV desplegado» al que promover: el pipeline promueve de QA a PROD, y DEV es donde se construye y se prueba antes del Pull Request;
- la paridad con QA la garantizan la misma imagen y el mismo esquema, no la misma máquina.

La configuración local puede incluir:

- PostgreSQL;
- servicios necesarios para desarrollo;
- datos seed;
- herramientas de inspección;
- mocks o servicios locales cuando corresponda.

Cuando una funcionalidad dependa específicamente de Auth, Storage, Realtime o RLS de Supabase, debe probarse también contra un entorno que reproduzca esas capacidades antes de considerarse validada.

El Docker Compose local y el esquema versionado viven en `MANI-API-Gateway` (perfil `db` para PostgreSQL y Adminer; perfil `web` para el cliente Flutter Web, cuya imagen `mani-web:local` se construye en `MANI-Frontend`). Ver §15.

5.4 Datos

Solo:

- seeds;
- fixtures;
- usuarios ficticios;
- datos sintéticos.

Nunca información real de PROD.

---

6. Ambiente QA

6.1 Objetivo

QA sirve para:

- integración multi-servicio;
- pruebas funcionales;
- pruebas de regresión;
- aislamiento multi-tenant;
- pruebas de carga;
- DAST;
- aceptación QA.

6.2 Supabase QA

Proyecto dedicado e independiente.

Debe disponer de:

- PostgreSQL;
- Auth;
- Storage;
- Realtime;
- RLS;
- esquema equivalente a PROD;
- datos sintéticos y reseteables.

6.3 Aplicaciones y servicios

Las imágenes se descargan desde GHCR hacia la máquina virtual del ambiente.

Los servicios se levantan con Docker Compose sobre esa VM (§9). Es el mecanismo vigente, no una alternativa provisional a otro ya decidido.

---

7. Ambiente PROD

7.1 Objetivo

PROD es el ambiente productivo oficial.

Contiene:

- usuarios reales;
- datos reales;
- documentos reales;
- secretos productivos;
- releases versionados.

7.2 Supabase PROD

Proyecto dedicado y aislado.

Debe contener:

- PostgreSQL;
- Auth;
- Storage;
- Realtime;
- políticas RLS productivas;
- backups;
- controles de acceso;
- cifrado en tránsito y en reposo según capacidades del proveedor.

7.3 Servicios

PROD utiliza únicamente imágenes promovidas y validadas previamente.

No se construye código directamente en PROD.

---

8. Registro de contenedores

El registro oficial es:

```
GitHub Container Registry — GHCR
```

Convención:

```
ghcr.io/trama-as/<servicio>:<tag>
```

Imágenes publicadas, una por repositorio desplegable:

```
ghcr.io/trama-as/mani-frontend
ghcr.io/trama-as/mani-api-gateway
ghcr.io/trama-as/mani-rules-service
ghcr.io/trama-as/mani-dispatch-service
ghcr.io/trama-as/mani-core-service
```

Cinco imágenes: `MANI-Docs` no se despliega y las disponibilidades van dentro de `mani-core-service`.

La infraestructura obtiene las imágenes desde GHCR.

---

9. Docker Compose en máquinas virtuales

Es el **mecanismo de despliegue vigente** de QA y PROD, no un apaño temporal mientras llega otra cosa: es lo que está decidido y lo que corre hoy.

```
GHCR
  ↓
pull de imágenes
  ↓
VM del ambiente (QA o PROD)
  ↓
docker compose up -d
```

Compose levanta los contenedores de cada VM. Los archivos viven en `MANI-API-Gateway`, un Compose por ambiente.

La definición debe:

- fijar imágenes por tag **y digest**, no por `latest`;
- inyectar configuración por ambiente desde fuera del archivo;
- no contener secretos;
- declarar la red interna por la que se comunican los servicios;
- declarar volúmenes cuando aplique;
- declarar health check por contenedor;
- declarar límites de CPU y memoria por contenedor;
- usar `restart: unless-stopped`, para que la caída de un contenedor no arrastre al resto;
- permitir rollback a una imagen anterior sin reconstruir.

Limitación conocida del modelo: una VM con Docker no da autoescalado ni réplicas gestionadas. La capacidad se añade por configuración y el techo es la VM. No es una decisión de arquitectura: es la consecuencia de que la orquestación siga abierta (§10).

---

10. Orquestación — decisión abierta

Kubernetes no está decidido ni desplegado. Es el orquestador exigido como objetivo por PROY-08, pero la decisión sigue abierta y el estado actual es Docker sobre VM.

10.1 Lo que sí está decidido

- Los artefactos son imágenes OCI y se obtienen desde GHCR.
- Azure/AKS **no** se adopta como proveedor obligatorio.
- QA y PROD corren hoy con Docker sobre una VM por ambiente (§9).
- La migración futura no debe requerir reconstruir las imágenes de aplicación.

10.2 Lo que está abierto

Nada de lo siguiente está definido, y ninguno se asume en ningún documento ni diagrama:

- si la plataforma final es Kubernetes gestionado, autogestionado u otra alternativa;
- proveedor de cómputo;
- número y tamaño de nodos;
- topología física;
- distribución por ambiente, por clúster o por namespace;
- ingress definitivo;
- estrategia de almacenamiento persistente.

Estas decisiones son `INFRA-01` e `INFRA-02` (§25) y **requieren ADR** antes de declararse definitivas. Hasta entonces:

- ningún documento declara un clúster como estado actual;
- no se fija proveedor ni dimensionamiento;
- no se documenta capacidad como definitiva;
- no se asume AKS ni Azure.

10.3 Por qué Compose no «equivale» a Kubernetes

Docker Compose resuelve composición y ciclo de vida de contenedores en **una** máquina. No da programación de carga entre nodos, autoescalado, ni recuperación ante la caída del host. Por eso el diseño no promete réplicas ni HPA mientras este sea el mecanismo vigente (SDD §7.6).

Lo que el modelo actual sí garantiza, y es lo que sostiene la promoción: la imagen es la misma, la configuración es externa y el rollback es por imagen anterior.

---

11. API Gateway

NGINX funciona como API Gateway.

Responsabilidades:

- TLS termination o integración con la capa que lo gestione;
- routing;
- CORS;
- rate limiting cuando se configure;
- validación inicial de JWT;
- propagación de `correlation_id`;
- controles transversales.

---

12. Servicios desplegables

Repositorio	Runtime	Responsabilidad principal
`MANI-Frontend`	Flutter / Dart	Cliente web y móvil
`MANI-API-Gateway`	NGINX	Punto de entrada y enrutamiento de APIs
`MANI-Rules-Service`	Java	Reglas de negocio por tenant
`MANI-Dispatch-Service`	.NET	Solicitudes, despacho y asignación
`MANI-Core-Service`	Node.js	Servicios core y disponibilidades

`MANI-Docs` es el sexto repositorio del proyecto y no se despliega.

Las disponibilidades no tienen contenedor, repositorio ni esquema propios: son un dominio del `MANI-Core-Service`, con sus tablas en el esquema `core` (SAD §7.3).

No se utiliza Serverpod como backend principal.

---

13. Supabase

Supabase se modela como plataforma administrada.

13.1 PostgreSQL

Motor relacional operacional.

Contiene:

- tablas;
- llaves;
- restricciones;
- índices;
- RLS.

El DDL oficial vive en el documento/modelo de datos y scripts versionados.

13.2 Auth

Responsable de:

- autenticación;
- gestión de credenciales;
- sesiones;
- emisión de JWT.

Cada ambiente utiliza identidades aisladas.

13.3 Storage

Documentos KYC en almacenamiento privado.

Convención lógica:

```
tenant_id/aliado_id/documento
```

13.4 Realtime

Utilizado para eventos y mensajería casi en tiempo real.

Realtime no contiene reglas de negocio.

---

14. Multi-tenancy

La seguridad se aplica en capas:

```
JWT firmado
   ↓
NGINX Gateway
   ↓
Servicio
   ↓
RLS PostgreSQL
```

Reglas:

- `tenant_id` autenticado proviene del token;
- no se confía en headers arbitrarios como fuente de autorización;
- cada servicio valida acceso;
- RLS refuerza aislamiento;
- Storage aplica controles equivalentes.

---

15. Esquema y migraciones

Principio **schema-first**:

1. el cambio se crea como migración/script versionado;
2. se valida en DEV/local;
3. se valida en QA;
4. se aplica en PROD.

Queda prohibido modificar manualmente el esquema productivo como procedimiento ordinario.

Los tres ambientes deben conservar compatibilidad estructural.

15.1 Dónde viven las migraciones

Las migraciones versionadas (`database/migrations/NNN_descripcion.sql` , idempotentes e inmutables una vez fusionadas), los scripts de `database/init/` y las verificaciones de `database/verify/` viven en el repositorio **`MANI-API-Gateway`** (ADR-0004, enmienda del 2026-10-07). Ya no están en `MANI-Frontend` .

Todo cambio de esquema se propone por PR a `MANI-API-Gateway` , con su referencia Jira.

15.2 Responsable por ambiente

Ambiente	Responsable	Cómo
DEV local	Cada desarrollador	`scripts/migrate-local.*` contra el contenedor `mani-postgres` (perfil `db` del Compose)
DEV / QA (Supabase)	DevOps	Aplica `database/migrations/` en orden, después de fusionar el PR. Automatizarlo en el pipeline de `MANI-API-Gateway` queda en CFG-29
PROD	DevOps, con aprobación del PR de release	Solo migraciones ya validadas en QA. Nunca cambios manuales al esquema

16. Qué se comparte y qué se aísla

Componente	Política
Código fuente	Compartido por repositorio/versionado
Imagen OCI	Compartida/promovida
DDL/migraciones	Compartidas
Definición de RLS	Compartida
Datos	Aislados
Usuarios/Auth	Aislados
Storage	Aislado
Secretos	Aislados
Dominios/URLs	Aislados
Logs	Aislados por ambiente
Configuración	Aislada
Supabase	Proyecto/entorno segregado
Registro GHCR	Compartido, tags/digests segregados

17. Secretos

DEV

Variables de desarrollo, nunca productivas.

QA

Secretos propios del ambiente.

PROD

Secretos exclusivos, acceso restringido.

Prohibiciones:

- credenciales en repositorio;
- secrets dentro de imágenes;
- reutilizar claves productivas en ambientes inferiores.

Secretos de CI

Los secretos que usan los pipelines se guardan como GitHub Actions secrets del repositorio o del environment, nunca en archivos versionados.

Repositorio	Secreto	Uso	Estado al 7 oct 2026
MANI-Core-Service	`SONAR_TOKEN`	análisis de SonarCloud del workflow `Quality Gate` (CFG-41)	**pendiente de crear** por DevOps

---

18. Red y TLS

Cada ambiente debe mantener su propia configuración de red y endpoint.

Ejemplos conceptuales:

```
DEV      → local / dominio dev
QA  → dominio QA
PROD     → dominio productivo
```

Los nombres concretos se definen cuando exista infraestructura asignada.

Requisitos:

- HTTPS para tráfico externo;
- secretos fuera del código;
- CORS por ambiente;
- reglas de red con mínimo privilegio.

---

19. Persistencia y archivos por ambiente

Capacidad	DEV	QA	PROD
PostgreSQL	Supabase DEV y/o local	Supabase dedicado	Supabase dedicado
Auth	DEV	QA	PROD
Storage	datos ficticios	datos sintéticos	documentos reales
Realtime	desarrollo	pruebas	productivo
Datos	ficticios	sintéticos	reales
RLS	misma definición	misma definición	misma definición

---

20. Observabilidad

Los componentes deben integrarse con:

- Prometheus;
- Grafana;
- Datadog.

Telemetría mínima:

- latencia;
- requests;
- errores;
- CPU;
- memoria;
- estado de servicios;
- logs estructurados;
- trazas;
- correlation ID.

Los ambientes deben poder distinguirse en etiquetas y dashboards.

---

21. Backups y recuperación

Para PROD se requiere:

- backups periódicos de la persistencia;
- verificación de restauración;
- retención acorde al proveedor y necesidades del proyecto;
- registro de incidentes;
- procedimiento de rollback de aplicación.

Los valores exactos de RPO/RTO deben mantenerse sincronizados con SAD/SDD cuando hayan sido ratificados.

No se fija aquí una cifra no aprobada.

---

22. Escalabilidad

Estado vigente — Docker sobre VM

La capacidad se añade por configuración del Compose y por el tamaño de la VM. Lo que el modelo actual sí da:

- health checks por contenedor;
- reinicio automático del contenedor caído (`restart: unless-stopped`);
- límites de CPU y memoria por servicio, para que uno no consuma la VM;
- rollback por imagen anterior.

Lo que **no** da, y por eso no se promete en ningún documento:

- programación de carga entre varias máquinas;
- autoescalado por métrica;
- recuperación ante la caída del host;
- rolling update sin ventana de indisponibilidad del servicio afectado.

Cuando se cierre la orquestación

Una vez resueltos INFRA-01 e INFRA-02, la plataforma podrá habilitar réplicas gestionadas, self-healing a nivel de nodo, rolling updates, balanceo y autoescalado cuando exista métrica y capacidad justificadas.

El número de réplicas no se fija hasta tener datos de carga, y el umbral de escalabilidad del SDD (§7.6) está enunciado como propiedad del artefacto precisamente para no depender de esa decisión.

---

23. Data Warehouse

El DW se mantiene separado del OLTP.

Flujo conceptual:

```
Supabase / PostgreSQL OLTP
        ↓
      CDC / ELT
        ↓
      Staging
        ↓
 Transformación / Calidad
        ↓
   Data Warehouse
        ↓
      BI / Métricas
```

El DW no escribe hacia el OLTP.

---

24. Promoción entre ambientes

La infraestructura recibe artefactos según la política DevOps:

```
DEV → QA → PROD
```

Infraestructura no redefine los Quality Gates.

La fuente de verdad para gates es `POLITICAS_DEVOPS_HERRAMIENTAS.md`.

24.1 Estado del análisis SonarCloud por repositorio

Esta tabla registra solo el estado de la configuración, no los umbrales: los umbrales viven en `POLITICAS_DEVOPS_HERRAMIENTAS.md` §14.1 y ADR-0005.

Un gate se considera **vinculante** solo cuando su check es obligatorio en un ruleset activo de GitHub y existe evidencia de un PR bloqueado. Mientras eso no ocurra, el estado es *preparado* o *activo sin bloqueo* (Políticas §6.2).

Repositorio	Proyecto SonarCloud	Configuración versionada	Check obligatorio en ruleset	Estado al 7 oct 2026
MANI-Core-Service	`Trama-AS_MANI-Core-Service` (organización `trama-as`), **por crear**	`sonar-project.properties` y script `test:coverage` (etapa 1, en revisión en MANI-Core-Service#16, sin fusionar); workflow `quality-gate.yml` preparado sin PR (etapa 2)	no; `develop` y `main` sin ruleset ni protección	**preparado, no activo** (CFG-41, SCRUM-1116)

Para MANI-Core-Service, la activación requiere, en orden: crear el proyecto con el análisis automático desactivado, asociar el Quality Gate y fijar el *new code baseline*, crear `SONAR_TOKEN`, fusionar el workflow, obtener un primer análisis en `develop` y agregar `Sonar Quality Gate` como check obligatorio en el ruleset de `develop` y `main`.

---

25. Decisiones abiertas

A la fecha quedan explícitamente abiertas:

INFRA-01 — Plataforma y hosting de orquestación

Pendiente decidir si la plataforma final es Kubernetes gestionado, autogestionado u otra alternativa, y con qué proveedor y dimensionamiento. Requiere ADR.

INFRA-02 — Topología final

Pendiente definir nodos, namespaces o clusters por ambiente, ingress y networking. Requiere ADR.

No están abiertas. Las siguientes decisiones están tomadas y no se vuelven a discutir sin un ADR que las revise:

Tema	Estado
GHCR como registro de imágenes	**definido**
Supabase de DEV compartido	**definido**
DEV en máquinas personales con Docker local	**definido**
Docker sobre VM, una por ambiente, para QA y PROD	**definido**
Tres ambientes: DEV, QA y PROD	**definido**
Kubernetes como plataforma	**abierto** — es el objetivo de PROY-08, no una decisión cerrada (§10)

Esta tabla es la respuesta a «¿esto ya está decidido?». Si un documento dice que GHCR o el modelo de ambientes están pendientes, está desactualizado y se corrige contra esta sección.

---

26. Trazabilidad

Este documento debe mantenerse alineado con:

- ADR-0004 — CI/CD y ambientes;
- ADR-0005 — DevSecOps;
- ADR-0006 — observabilidad;
- ADR-0012 — Supabase/PostgreSQL + RLS;
- ADR-0013 — documentos KYC;
- ADR-0015 — pruebas multi-tenant;
- ADR-0018 — identidad de tenant;
- ADR-0019 — SOA + Gateway + arquitectura políglota;
- SRS — RNF y restricciones;
- SAD;
- SDD;
- Modelo de Datos.

Cuando exista una nueva decisión de hosting/clúster, este documento se actualiza después del ADR correspondiente.